COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CpE21S2
DATE PERFORMED:	9/16/2026	**DATE SUBMITTED:**	9/20/2026
NAME:	Fernandez, Cedric S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 8.1 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

Table 8-1 (Array of Values for Sort Algorithm Testing)

```cpp
#include <iostream>
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "SortingAlgo.h"

const int max_size = 100;

int main() {


    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i <max_size ; i++){
        dataset[i] = rand();
        std::cout<< dataset[i] <<" ";
    }
    std::cout<<std::endl;


    return 0;
}
```

Output

```
100 item integer (unsorted):
7625 3644 16787 29610 10878 19051 23126 25777 15
286 10009 23435 17041 2014 29322 17850 2496 2441
8 10767 5440 10293 3882 25209 22772 18813 19557
5331 7178 10508 3461 9041 10040 24273 29834 1088
3 23832 23053 11908 29390 13774 32356 27731 5661
 7658 5941 22706 11732 15504 5415 12148 7430 258
14 18948 23052 19175 13933 5835 13651 24964 1267
6 16384 25734 25777 16873 23623 21902 13635 1393
 4401 23173 21632 20891 4694 31861 31700 15472 5
796 27491 23750 11437 14326 12172 7465 13604 289
04 32750 25495 16224 11498 25098 2075 18824 7828
 30441 15662 15237 20538 6493 17637 5991 8895
```

Observation & Analysis

Best case: **O(n)**
Worst case: **O(n)**
Both the worse and best case would be O(n) because no matter the following loop will always generate 100 items for the array

Table 8-2 (Shell Sort Technique)

main.cpp

```cpp
#include <iostream>
#include <cstdlib> // For generating random integers
#include <ctime>   // For time-based seeding
#include <algorithm> // For std::copy
#include "SortingAlgo.h"

const int max_size = 100;
void displayArr(int arr[]);

int main() {

    std::cout<<"100 item integer (unsorted): "<<std::endl;
    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size ; i++){
        dataset[i] = rand();
    std::cout<< dataset[i] <<" ";
}
std::cout<<std::endl;

    int shellArray[max_size];
    int mergeArray[max_size];
    int quickArray[max_size];

std::copy(dataset, dataset + max_size, shellArray);
    shellSort(shellArray, max_size);
    std::cout << "\nShell Sort:\n";
    for (int i = 0; i < max_size; i++) {
        std::cout << shellArray[i] << " ";
    }
    std::cout << " " << std::endl;

    return 0;
}
```

sortingAlgo.h

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

template <typename T>
void shellSort(T array[], int size) {
    for (int interval = size / 2; interval > 0; interval /= 2) {
        for (int i = interval; i < size; i++) {
            T temp = array[i];
            int j = i;
            while (j >= interval && array[j - interval] > temp) {
                array[j] = array[j - interval];
                j -= interval;
            }
            array[j] = temp;
        }
    }
}
#endif
```

Output

```
100 item integer (unsorted):
11916 4042 28421 27684 31045 1430 16887 13194 19099 25144 25694 9382 27281 13243 1
831 10987 4893 5959 18941 21486 28880 18234 25918 28186 5816 20345 22738 32677 270
4 23561 21121 10690 17900 21977 23109 19481 26185 16866 1288 31860 19336 5746 1758
 9915 11589 31429 31062 26914 13618 9430 10205 161 2490 18804 31140 22949 27620 21
44 27028 6405 27769 20470 16991 12599 31849 21970 21000 22468 23659 15893 30373 28
09 12260 32672 26724 14303 14090 4064 25804 24686 29769 675 8005 16894 16614 23594
6875 25598 5346 18815 20799 5864 1569 13065 19214 15220 17099 23901 3414 2573

Shell Sort:
161 675 1288 1430 1569 2490 2573 3414 4042 4064 4893 5346 5746 5816 5864 5959 6405
6875 8005 9382 9430 9915 10205 10690 10987 11589 11916 12260 12599 13065 13194 132
3 13618 14090 14303 14831 15220 15893 16614 16866 16887 16894 16991 17099 17582 17
00 18234 18804 18815 18941 19099 19214 19336 19481 20345 20470 20799 21000 21121 2
144 21486 21970 21977 22468 22738 22949 23109 23561 23594 23659 23901 24686 25144
5598 25694 25804 25918 26185 26724 26914 27014 27028 27281 27620 27684 27769 28186
28409 28421 28880 29769 30373 31045 31062 31140 31429 31849 31860 32672 32677
```

Observation & Analysis

Best case: The following time complexity would be the best case **O(n log n)** assuming the elements are mostly in the correct position so fewer elements move between intervals.

Worst case: Shell sort is slow when it comes to many element making it so that it needs to move multiple element this the time complexity would be **O(n^2)** cause each element would need to be moved in different interval

Table 8-3 (Merge Sort Technique)

main.cpp

```cpp
#include <iostream>
#include <cstdlib> // For generating random integers
#include <ctime>   // For time-based seeding
#include <algorithm> // For std::copy
#include "SortingAlgo.h"

const int max_size = 100;

int main() {

    std::cout<<"100 item integer (unsorted): "<<std::endl;
    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size ; i++){
        dataset[i] = rand();
    std::cout<< dataset[i] <<" ";
}
std::cout<<std::endl;
    int mergeArray[max_size];
std::copy(dataset, dataset + max_size, mergeArray);
    mergeSort(mergeArray, 0, max_size - 1);
    std::cout << "\nMerge Sort:\n";
    for (int i = 0; i < max_size; i++) {
        std::cout << mergeArray[i] << " ";
    }
    std::cout << " " << std::endl;

    return 0;
}
```

sortingAlgo.h

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

template <typename T>
void merge(T Arr[], int left, int middle, int right) {

    int N1 = middle - left + 1;
    int N2 = right - middle;

    T* Left = new T[N1];
    T* Right = new T[N2];

    for (int i = 0; i < N1; i++) {
        Left[i] = Arr[left + i];
    }
    for (int j = 0; j < N2; j++) {
        Right[j] = Arr[middle + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while (i < N1 && j < N2) {

        if (Left[i] <= Right[j]) {
            Arr[k] = Left[i];
            i++;
        }
        else {
            Arr[k] = Right[j];
            j++;
        }
        k++;
    }
    while (i < N1) {
        Arr[k] = Left[i];
        i++;
        k++;
    }
    while (j < N2) {
        Arr[k] = Right[j];
        j++;
        k++;
    }
    delete[] Left;
    delete[] Right;
}

template <typename T>
void mergeSort(T array[], int left, int right) {
    if (left < right) {
        int middle = left + (right - left) / 2;
        mergeSort(array, left, middle);
        mergeSort(array, middle + 1, right);
        merge(array, left, middle, right);
    }
}
#endif
```

Output

```
100 item integer (unsorted):
14897 19705 19945 10161 11068 14822 7739 27568 20901 15286 21728 16829 22218 13
991 25651 13720 12024 18204 19220 3702 31012 29322 4663 9985 1756 24468 13250 1
697 8317 22901 8472 8186 22949 8863 23279 22710 2539 23201 20393 16004 15822 31
566 29665 3424 13292 12572 18756 27414 14965 4810 18338 31745 222 31814 23620 1
3593 12762 4301 29569 3885 27014 10324 16524 32620 23523 11303 14224 22826 1626
6 9936 28159 31969 20635 2874 21799 30838 32759 30486 774 31934 8879 29822 3209
5 19521 13756 32472 15117 1854 2544 20222 20376 30959 27692 12956 29958 18957 7
636 29401 26611 15511

Merge Sort:
222 774 1697 1756 1854 2539 2544 2874 3424 3702 3885 4301 4663 4810 7636 7739 8
186 8317 8472 8863 8879 9936 9985 10161 10324 11068 11303 12024 12572 12762 129
56 13250 13292 13593 13720 13756 13991 14224 14822 14897 14965 15117 15286 1551
1 15822 16004 16266 16524 16829 18204 18338 18756 18957 19220 19521 19705 19945
 20222 20376 20393 20635 20901 21728 21799 22218 22710 22826 22901 22949 23201
23279 23523 23620 24468 25651 26611 27014 27414 27568 27692 28159 29322 29401 2
9569 29665 29822 29958 30486 30838 30959 31012 31566 31745 31814 31934 31969 32
095 32472 32620 32759
```

Observation & Analysis

Best case: the following would be **O(n log n)** because the merge sort would divide the array into smaller parts repeatedly until every sub array only has one element and even if the array is lets say already or almost sorted
Worst case: **O(n log n)** For the worst case time complexity would still be the same because regardless if the array is messy or almost sorted, merge sort would still need to divide every array and sub array until one element remains just like for the best case.

Table 8-4 (Quick Sort Technique)

main.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <algorithm>
#include "SortingAlgo.h"

const int max_size = 100;

int main() {

    std::cout<<"100 item integer (unsorted): "<<std::endl;
    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size ; i++){
        dataset[i] = rand();
    std::cout<< dataset[i] <<" ";
}
std::cout<<std::endl;

    int quickArray[max_size];

std::copy(dataset, dataset + max_size, quickArray);

    quickSort(quickArray, 0, max_size - 1);
    std::cout << "\nQuick Sort:\n";
    for (int i = 0; i < max_size; i++) {
        std::cout << quickArray[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

sortingAlgo.h

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

template <typename T>
int partition(T array[], int low, int high) {
    T pivot = array[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (array[j] < pivot) {
            i++;
            T temp = array[i];
            array[i] = array[j];
            array[j] = temp;
        }
    }
    T temp = array[i + 1];
    array[i + 1] = array[high];
    array[high] = temp;
    return i + 1;
}

template <typename T>
void quickSort(T array[], int low, int high) {
    if (low < high) {
        int pivot = partition(array, low, high);
        quickSort(array, low, pivot - 1);
        quickSort(array, pivot + 1, high);
    }
}

#endif
```

Output

```
100 item integer (unsorted):
22623 22466 15855 25946 3191 32121 31766 15684 27868 17950 32230 22739 6766 483
8 5204 10107 24548 30843 25325 23008 6497 22597 9232 25914 14920 7912 32091 220
14 25938 19470 30819 13755 19166 10086 29608 2113 16776 18157 8136 9225 19708 1
9052 23794 27089 1805 28737 3230 8071 7937 26467 11131 29288 25391 673 18345 25
705 2253 12586 10025 10454 9731 24658 12946 13369 4675 10899 30725 893 8413 306
74 23138 24147 27086 29412 16572 6357 5337 30801 23051 6249 23258 25176 18721 8
169 12630 22067 19680 9985 21590 30364 14505 16600 8822 8079 20759 22002 32569
3922 24814 13653

Quick Sort:
673 893 1805 2113 2253 3191 3230 3922 4675 4838 5204 5337 6249 6357 6497 6766 7
912 7937 8071 8079 8136 8169 8413 8822 9225 9232 9731 9985 10025 10086 10107 10
454 10899 11131 12586 12630 12946 13369 13653 13755 14505 14920 15684 15855 165
72 16600 16776 17950 18157 18345 18721 19052 19166 19470 19680 19708 20759 2159
0 22002 22014 22067 22466 22597 22623 22739 23008 23051 23138 23258 23794 24147
 24548 24658 24814 25176 25325 25391 25705 25914 25938 25946 26467 27086 27089
27868 28737 29288 29412 29608 30364 30674 30725 30801 30819 30843 31766 32091 3
2121 32230 32569
```

Observation & Analysis

Best case: The best case would be **O(n log n)** because if the pivot makes it so that the array can be split evenly equal parts for the left and right parts which would be considered the quickest situation for quick sort
Worst case: The worst case for the following would be **O(n^2)** because a quick sort would be very slow if each pivot would create uneven and unbalanced parts each time.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes we can after quick sort from the partition the left and right sub list can be sorted using other sorting algorithms this often called hybrid sorting which is demonstrated below from the geeksforgeeks.org that in which they used a quick sort for the larger portion of an array then switched to insertion for the smaller cut out parts of the subarray.

```cpp
#include<bits/stdc++.h>
using namespace std;

// Function to perform the insertion sort
void insertion_sort(int arr[], int low, int n)
  {
    for(int i=low+1;i<n+1;i++)
    {
      int val = arr[i] ;
      int j = i ;
    while (j>low && arr[j-1]>val)
      {
        arr[j]= arr[j-1] ;
      j-= 1;
      }
    arr[j]= val ;
    }
  }

//The following two functions are used
// to perform quicksort on the array.

// Partition function for quicksort
int partition(int arr[], int low, int high)
{
  int pivot = arr[high] ;
  int i ,j;
  i = low;
  j = low;
  for (int i = low; i < high; i++)
    {
```

```cpp
      arr[j] = temp;
      j += 1;
      }
    }
    int temp = arr[j];
    arr[j] = arr[high];
    arr[high] = temp;
  return j;
  }

// Function to call the partition function
// and perform quick sort on the array
void quick_sort(int arr[], int low,int high)
{
  if (low < high)
  {
    int pivot = partition(arr, low, high);
    quick_sort(arr, low, pivot-1) ;
    quick_sort(arr, pivot + 1, high) ;
  }
}

//======Hybrid function -> Quick + Insertion sort======
void hybrid_quick_sort(int arr[], int low, int high)
{
  while (low < high)
    {
    // If the size of the array is less
    // than threshold apply insertion sort
```

```
69          break;
70        }
71        else
72          {
73            int pivot = partition(arr, low, high) ;
74
75          // Optimised quicksort which works on
76          // the smaller arrays first
77
78          // If the left side of the pivot
79          // is less than right, sort left part
80          // and move to the right part of the array
81
82          if (pivot-low<high-pivot)
83            {
84            hybrid_quick_sort(arr, low, pivot - 1);
85            low = pivot + 1;
86          }
87          else
88            {
89
90            // If the right side of pivot is less
91            // than left, sort right side and
92            // move to the left side
93
94            hybrid_quick_sort(arr, pivot + 1, high);
95            high = pivot-1;
96            }
97        }
98      }
99    }
100   // Driver Code
101   int main()
102   {
103     int  arr[21] = { 24, 97, 40, 67, 88, 85, 15,
104      66, 53, 44, 26, 48, 16, 52,
105      45, 23, 90, 18, 49, 80, 23 };
106   hybrid_quick_sort(arr, 0, 20);
107     for(int i = 0; i < 21; i++)
108       cout << arr[i] << ", ";
109   }
110   // This code is contributed by ishayadav2918
```

OUTPUT OF ARRAY { 24, 97, 40, 67, 88, 85, 15, 66, 53, 44, 26, 48, 16, 52, 45, 23, 90, 18, 49, 80, 23 }

```
15, 16, 18, 23, 23, 24, 26, 40, 44, 45, 48, 49, 52, 53, 66, 67, 80, 85, 88, 90, 97,
PS C:\Users\csf18\Desktop\Hands 8\Sorting Techniques 8.1>
```

REFERENCE: https://www.geeksforgeeks.org/dsa/advanced-quick-sort-hybrid-algorithm/

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- Quick and merge sort can give us the fastest performance for this specific array because they have an O(n log n) time complexity for this situation because they repeatedly divide the array into half giving us an log n time complexity then after the elements are sorted for each level that would give us an o(n) and which in the end would give us an O(n log n) for their time complexity.

CONCLUSION

Overall this activity helped me learn more about different algorithm techniques and how each shell, merge, and quick sort works and how they can be implemented when it comes to a program. It also gave me an insight into what are the advantages and disadvantages when it comes their time complexity and which one is the fastest and slowest when it comes to different situations and how different sorting algorithms can be used to work with each other for sorting which I come to know now as hybrid sorting which is often used to to maximize performance, adaptability, and efficiency across different data sizes .

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept explanation, code assistance, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explanation of concept, assistance of trouble shooting**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.